



PRD — Perccent Unified Calculator (SIP / Lumpsum / Step-up / SWP / EMI)



Audience: an agentic AI coding agent building the calculator end-to-end. This PRD is the source of truth. Follow it literally. Where it says **MUST**, treat it as a hard invariant; **SHOULD** is a strong default that may be overridden only with an explicit reason. Do not invent financial assumptions — every assumption is specified here or must be surfaced to the user in the UI.



Product thesis: the best calculator gives the customer *every* relevant customization option, stacked and simultaneously available, and relays *all* the information behind the result (formula used, assumptions, year-by-year breakdown, charts). Transparency and configurability are the product.

1. Objective

Build a single, unified financial calculator web app that supersedes the current `percent-calculator.vercel.app`. It **MUST** cover five calculation modes sharing one tested calculation engine:

1. **SIP** (Systematic Investment Plan)
2. **Lumpsum** (one-time investment growth)
3. **Step-up SIP** (annual escalation)
4. **SWP** (Systematic Withdrawal Plan)
5. **EMI** (loan installment — reducing & flat)

2. Principles (non-negotiable)

- **Full customization, stacked:** all advanced options (inflation, expense ratio, exit load, taxes, step-up, timing convention, compounding cadence) are available in every relevant mode via an always-accessible “Advanced” panel. Nothing is hidden behind paywalls or separate pages.

- **Total transparency:** every result shows the exact formula/method used, the resolved periodic rate, the assumptions applied, and a year-by-year (and optionally month-by-month) breakdown.
- **One engine, many UIs:** all five modes call a single pure, deterministic, unit-tested calculation library. No formula logic in UI components.
- **Estimates, clearly labelled:** every output carries a SEBI-appropriate disclaimer. The tool never implies guaranteed returns.
- **No silent magic:** defaults are explicit and visible; changing any toggle recomputes instantly and updates the “method used” label.

3. Tech expectations

- **Stack:** TypeScript + React (Next.js App Router SHOULD be used). Keep the existing Vercel deployment target.
- **Calc engine:** a standalone, framework-free TS module (`/lib/calc/`) with **zero UI imports**, exported as pure functions. 100% unit-test coverage on the engine is **REQUIRED**.
- **Charts:** a lightweight charting lib (Recharts or similar). Donut for invested-vs-returns, line/area for growth & amortization.
- **State:** URL-encoded state so any configuration is shareable via link (deep-linking is **REQUIRED**).
- **Money math:** compute in floating point internally but **round only at display**; never round intermediate periodic balances. Currency display uses the **Indian numbering system** (lakh/crore) with ₹ and `Intl.NumberFormat('en-IN')`.

4. Calculation engine specification (the core)

4.1 Shared conventions & invariants

- **Rate conversion (MUST default to effective):**
 - Effective (default): $i = (1 + \text{annualRate})^{(1/m)} - 1$
 - Nominal (toggle): $i = \text{annualRate} / m$
 - where m = periods per year (12 for monthly). The UI MUST expose this toggle and label which was used. **Invariant:** with the effective method, $(1 + i)^m$ MUST equal $(1 + \text{annualRate})$ within 1e-9.
- **Compounding cadence:** selectable {monthly (default), quarterly, annually, daily}. Drives m .

- **Contribution/withdrawal timing:** {begin-of-period (annuity-due, default for SIP), end-of-period}. Drives the trailing `×(1+i)`.
- **Determinism:** identical inputs MUST always produce identical outputs (no `Date.now/random` in engine).

4.2 SIP & Step-up SIP

Closed form (flat SIP, annuity-due):

```
FV = P * ((Math.pow(1 + i, n) - 1) / i) * (1 + i)
// ordinary annuity: drop the trailing (1 + i)
```

Iterative engine (REQUIRED — used whenever step-up, inflation, expense ratio, exit load, or tax is enabled):

```
let balance = 0
let invested = 0
let contribution = P
for (let month = 1; month <= n; month++) {
  if (timing === 'begin') balance += contribution
  balance *= (1 + iNetOfExpense) // expense ratio reduces i (see 4.5)
  if (timing === 'end') balance += contribution
  invested += contribution
  if (stepUpEnabled && month % 12 === 0) contribution *= (1 + stepUpPct)
}
```

- **Step-up:** escalate contribution by `stepUpPct` every 12 months (configurable cadence). Default escalation point = month 13, 25, 37...
- Outputs: `invested`, `maturityValue (gross)`, `estimatedReturns = maturity - invested`.

4.3 Lumpsum

```
FV = P * Math.pow(1 + i, n) // single contribution at t0
```

4.4 SWP

Iterative (REQUIRED — no closed form once withdrawals interact with growth):

```
let balance = corpus
let totalWithdrawn = 0
```

```

let depletionMonth = null
for (let month = 1; month <= n; month++) {
  if (timing === 'begin') { /* withdraw first */ balance -= W; totalWithdrawn += W }
  balance *= (1 + i)
  if (timing === 'end') { balance -= W; totalWithdrawn += W }
  if (balance <= 0 && depletionMonth === null) { depletionMonth = month; balance = 0; break }
}

```

- **Step-up SWP** SHOULD be supported (escalate `W` annually).
- **Sustainable-withdrawal solver (REQUIRED)**: given corpus, rate, tenure → solve for the max `W` that leaves `balance ≥ 0` at end (binary search).
- **Longevity solver (REQUIRED)**: given corpus, rate, `W` → return `depletionMonth` (or “never depletes” if growth ≥ withdrawal).
- Outputs: `totalWithdrawn`, `closingCorpus`, `longevity (months/years)`, month-by-month schedule.

4.5 Real-world adjustment layers (all toggleable, stackable)

Layer	Effect on math	Default
Inflation	Compute real value: <code>real = nominal / (1 + inflation)^years</code> . Show BOTH nominal & real.	off
Expense ratio (TER)	Reduce the gross annual return before rate conversion: <code>annualNet = annualRate - ter</code> .	off
Exit load	Apply a one-time % haircut on redemption value if held < lock-in period.	off
LTCG tax	On redemption, tax gains above the ₹1.25L/yr exemption at 12.5% (equity LTCG, FY25-26 rules). Surface as a separate line, never silently.	off

The agent MUST keep these as composable functions applied in a defined order:
TER → growth → exit load → tax → inflation-deflation for display.

4.6 EMI

```

// Reducing balance (DEFAULT)
const R = annualRate / (12 * 100)
const EMI = P * R * Math.pow(1 + R, N) / (Math.pow(1 + R, N) - 1)
const totalInterest = EMI * N - P

```

```
// Flat rate (toggle / comparison)
const flatInterest = P * (annualRate / 100) * years
const flatEMI = (P + flatInterest) / N
```

- **Amortization schedule (REQUIRED):** per-month principal, interest, balance.
- **Prepayment (SHOULD):** support lump-sum prepayment at month X and/or EMI increase; recompute schedule.
- **Flat-vs-reducing comparison view (REQUIRED):** show both EMIs and the effective-rate gap side by side.

5. Inputs specification

Mode	Input	Type	Range / validation	Default
SIP	Monthly amount	₹	100 – 10,00,00,000	5,000
SIP	Expected return p.a.	%	0 – 40	12
SIP	Time period	yrs	1 – 50	10
Step-up	Annual step-up	%	0 – 50	10
Lumpsum	Amount	₹	500 – 10,00,00,000	1,00,000
SWP	Corpus	₹	1,000 – 100,00,00,000	10,00,000
SWP	Monthly withdrawal	₹	100 – corpus	8,000
SWP	Expected return p.a.	%	0 – 40	8
SWP	Tenure	yrs	1 – 50	20
EMI	Loan amount	₹	1,000 – 100,00,00,000	10,00,000
EMI	Interest rate p.a.	%	0 – 36	9
EMI	Tenure	yrs/mo	1 mo – 40 yrs	20 yrs
Advanced (all)	Rate method	enum	effective / nominal	effective
Advanced (all)	Compounding	enum	monthly/qtr/annual/daily	monthly
Advanced (invest)	Timing	enum	begin / end	begin
Advanced	Inflation, TER, exit load, LTCG	%/toggle	sane bounds	off

Validation rules (MUST): reject negatives where nonsensical; clamp to ranges with inline messaging; handle `rate = 0` (use `FV = P × n` for SIP, `EMI = P / N` for loans — avoid divide-by-zero); guard very long tenures against overflow.

6. Outputs & information relayed (transparency requirements)

Every mode MUST display:

1. **Headline numbers** (e.g., Invested / Est. returns / Total value; or EMI / Total interest / Total payable; or Total withdrawn / Closing corpus / Longevity).
 2. **"Method used" badge:** e.g., "Effective monthly rate 0.949% · annuity-due · monthly compounding".
 3. **Donut chart** (invested vs returns) and a **growth/area line chart** over time.
 4. **Year-by-year table** (expandable to month-by-month) with opening balance, contribution/withdrawal, growth, closing balance.
 5. **Assumptions panel:** plain-language list of every assumption currently applied.
 6. **Real vs nominal** values when inflation is on.
 7. **Disclaimer** line aligned to SEBI/RIA norms.
 8. **Shareable link + Download PDF/PNG** of the result.
-

7. UI / UX specification

- **Layout:** left = inputs (sliders **and** numeric fields, kept in sync); right = results + charts. Mode switcher as tabs.
- **Stacked customization:** an always-visible "Advanced options" section (not a separate page) exposing rate method, compounding, timing, inflation, TER, exit load, tax, step-up.
- **Real-time recompute:** debounced ($\leq 100\text{ms}$) on every input change; no "Calculate" button required (but provide one for accessibility).
- **Tooltips on every field** explaining the concept (definitions of annuity-due, effective vs nominal, TER, LTCG, etc.).
- **Comparison mode:** allow comparing two configurations side by side (e.g., effective vs nominal; flat vs reducing; with vs without step-up).
- **Responsive & accessible:** mobile-first, keyboard navigable, ARIA labels, WCAG AA contrast.

- **i18n number formatting:** Indian comma grouping and lakh/crore abbreviations on axes.

8. Edge cases & invariants (MUST handle)

- `rate = 0` → linear results, no NaN/Infinity.
- `tenure` very large (50 yrs daily compounding) → no overflow; cap iterations sensibly.
- SWP where withdrawal < growth → “corpus never depletes” (longevity = ∞), don’t loop forever (cap at tenure).
- SWP where withdrawal > corpus in month 1 → depletes month 1.
- Rounding: intermediate balances unrounded; display rounded to ₹ (0 decimals) by default.
- **Cross-mode invariant:** SIP with step-up = 0 MUST equal flat SIP. SWP with W = 0 MUST equal lumpsum growth. EMI reducing with rate → 0 MUST approach `P / N`.

9. Acceptance tests (engine — REQUIRED, with tolerances)

#	Scenario	Expected ($\approx$, tolerance $\pm 0.5\%$)
T1	SIP ₹5,000/mo, 12% p.a., 10 yrs, effective , annuity-due	Invested ₹6,00,000 · Maturity $\approx$ ₹11,20,000
T2	Same as T1 but nominal ($\div 12$)	Maturity $\approx$ ₹11,61,700 (proves nominal overstates $\sim 3.7\%$)
T3	Lumpsum ₹1,00,000, 12% p.a., 10 yrs, effective	$\approx$ ₹3,10,585
T4	EMI ₹10,00,000, 9% p.a., 20 yrs, reducing	EMI $\approx$ ₹8,997 · Total interest $\approx$ ₹11,59,300
T5	EMI same but flat	Flat interest = ₹18,00,000 , flat EMI = ₹11,667 (must exceed reducing)
T6	SWP corpus ₹12,00,000, 0% return , withdraw ₹10,000/mo	Longevity = exactly 120 months
T7	SWP corpus ₹10,00,000, 8% p.a., withdraw ₹5,000/mo	Corpus grows (longevity = never within tenure)
T8	Effective-rate invariant	<code>(1 + i)^12 == 1.12</code> within 1e-9 for 12%

The agent MUST also spot-check T1/T3 against Groww and the SEBI calculator and document any residual delta.

10. Deliverables

1. `/lib/calc/` — pure TS engine: `sip.ts`, `lumpsum.ts`, `swp.ts`, `emi.ts`, `rates.ts`, `adjustments.ts`, plus `index.ts` barrel.
2. `/lib/calc/__tests__/` — unit tests covering Section 9 + invariants.
3. UI: mode tabs, input panel (sliders+fields), advanced panel, results, charts, schedule table, comparison mode.
4. URL state encode/decode util + shareable links.
5. PDF/PNG export.
6. `README` documenting every formula, default, and assumption (mirror Section 4).
7. Accessibility & responsive pass.

11. Out of scope (v1)

- Live NAV / backtested real-fund SWP (note as a fast-follow; design the engine so a NAV provider can be plugged into the `growth` step later).
- Auth, user accounts, saved scenarios beyond shareable URLs.
- Multi-currency (₹ only for v1).



Compliance: all investment outputs MUST carry: "These are illustrative estimates based on the assumptions shown and are not guaranteed returns. Mutual fund investments are subject to market risks; read all scheme-related documents carefully." EMI outputs MUST note that actual lender terms, fees, and floating-rate resets may differ.